

BLOQUE 2 · ENTORNOS DE DESARROLLO — FASE DE PRUEBAS

2.1 Metodologías de Prueba

Tipo	También llamada	Quién la realiza	Qué se verifica
Caja Blanca	Estructural / Clear-box	Tester con acceso al código fuente	Flujos de control, bucles y condiciones. Cada línea debe ejecutarse al menos una vez.
Caja Negra	Funcional / Behavioural	Tester que desconoce el código interno	Que la salida (output) sea la correcta para cada entrada (input) definida en los requisitos.

Técnicas de Caja Blanca

Técnica	Definición
Prueba de cubrimiento	Verifica que cada línea o instrucción del código se ejecute al menos una vez durante las pruebas. El objetivo es alcanzar el mayor porcentaje de cobertura posible del código fuente.
Prueba de condiciones	Comprueba todas las condiciones lógicas del código (IF, WHILE, SWITCH...). Se diseñan casos para que cada condición tome valor verdadero y falso al menos una vez.
Prueba de bucles	Se centra específicamente en los bucles (FOR, WHILE, DO-WHILE). Se prueban los casos límite: cero iteraciones, una iteración, el número máximo y valores intermedios.

Técnicas de Caja Negra

Técnica	Definición
Clases de equivalencia	Divide los datos de entrada en grupos (clases) que el sistema debería tratar de forma idéntica. Se prueba un representante de cada clase, tanto válida como inválida, en lugar de probar todos los valores posibles.
Valores límite	Prueba los valores en los extremos de cada clase de equivalencia, ya que la mayoría de errores se concentran en los bordes. Por ejemplo, si un campo acepta de 1 a 100: se prueban 0, 1, 100 y 101.
Prueba de interfaz	Verifica la correcta comunicación entre módulos, sistemas o componentes. Comprueba que los datos se transmiten correctamente a través de las interfaces (APIs, formularios, integraciones).

2.2 Diseño de Casos de Prueba

Un **caso de prueba** es un conjunto de condiciones y datos de entrada bajo los cuales se determina si una funcionalidad del sistema opera según sus especificaciones.

Elemento	Pregunta clave	Detalle
Objetivo	¿Qué validamos?	Funcionalidad concreta que se evalúa.
Procedimiento	¿Cómo lo probamos?	Secuencia de pasos exactos y reproducibles.
Datos de entrada	¿Con qué valores?	Valores que se introducen al sistema.
Resultado esperado	¿Qué debería pasar?	Comportamiento correcto según los requisitos.
Resultado obtenido	¿Qué pasó realmente?	Lo que el sistema hace. Si difiere del esperado → se reporta un bug.

2.3 SCRUM

Marco de trabajo ágil basado en ciclos cortos llamados **Sprints** (1–4 semanas). Al final de cada Sprint se entrega un incremento funcional del producto.

Roles

Rol	Función principal
Product Manager (PM)	Visión estratégica del producto a largo plazo. Define roadmap, analiza mercado y comunica la estrategia. Se sitúa entre los clientes y el Product Owner.
Product Owner (PO)	Maximiza el valor del producto. Gestiona y prioriza el Product Backlog. Traduce la estrategia del PM en requisitos concretos para el equipo.
Scrum Master	Facilita el proceso Scrum, elimina impedimentos y protege al equipo de interrupciones externas.
Development Team	Equipo multifuncional (devs, testers, diseñadores) que ejecuta el trabajo del Sprint. Autoorganizado.

*Nota: entre los clientes/stakeholders y el Product Owner puede existir la figura del **Product Manager**, que actúa como capa estratégica. El PM define qué construir; el PO asegura que se construya correctamente.*

Eventos (Ceremonias)

Evento	¿Cuándo?	Objetivo
Sprint Planning	Inicio del Sprint	El equipo selecciona y planifica las tareas del Product Backlog para el Sprint.
Daily Scrum	Cada día (15 min)	Sincronización: ¿Qué hice? ¿Qué haré? ¿Tengo impedimentos?
Sprint Review	Fin del Sprint	El equipo presenta el incremento a los stakeholders y recoge feedback.
Sprint Retrospective	Tras la Review	El equipo reflexiona sobre el proceso para mejorar en el siguiente Sprint.

Artefactos

Artefacto	Descripción
Product Backlog	Lista priorizada de todo lo que el producto necesita. La gestiona el Product Owner.
Sprint Backlog	Subconjunto del Product Backlog seleccionado para el Sprint actual.
Incremento	Versión funcional y potencialmente entregable del producto al final de cada Sprint.

2.4 Casos de Uso y Casos de Prueba

Aspecto	Caso de Uso	Caso de Prueba
¿Qué es?	Describe cómo el sistema interactúa con un actor para lograr un objetivo.	Verifica que el sistema funcione según lo especificado.
Perspectiva	Orientado a requisitos funcionales.	Orientado a la detección de errores.
¿Quién lo crea?	Analista / Product Owner.	Tester / QA Engineer.
¿Cuándo?	Fase de análisis y diseño.	Fase de pruebas (también antes con TDD).
Contenido	Actor, precondiciones, flujo principal, flujos alternativos, postcondiciones.	Objetivo, pasos, datos de entrada, resultado esperado y resultado obtenido.
Ejemplo	«El usuario inicia sesión con sus credenciales.»	«Email válido + contraseña incorrecta → mostrar mensaje de error.»

¿Cuál se hace primero? El caso de uso debe confeccionarse antes. Define el comportamiento esperado del sistema desde la perspectiva del usuario y los requisitos. El caso de prueba depende de él: sin saber qué debe hacer el sistema (caso de uso), no se puede definir qué verificar ni qué resultado esperar. El caso de uso es la fuente de verdad; el caso de prueba es su validación.

Relación entre ambos: un solo caso de uso puede generar múltiples casos de prueba (flujo principal, flujos alternativos, valores límite, escenarios negativos). Existe una relación de 1 a N entre caso de uso y casos de prueba.

2.5 Fases de un Producto

Fase	Público principal	Descripción
Pre-Alpha	Solo equipo interno	Versión incompleta en desarrollo activo. Funcionalidades básicas en construcción, muchos errores críticos.
Alpha	Equipo interno y QA	Primera versión completa pero muy inestable. Pruebas intensivas internas. No apta para usuarios finales.
Beta	Usuarios externos voluntarios	Producto funcional con la mayoría de características implementadas. Se detectan errores residuales con feedback externo.

Beta Cerrada	Grupo selecto por invitación	Igual que Beta pero con acceso restringido para un control más preciso del feedback.
Release Candidate (RC)	Usuarios avanzados	Candidata a versión final. Solo se corrigen bugs críticos. Sin nuevas funcionalidades.
Gold / RTM	Todos los usuarios (producción)	Versión final aprobada para distribución masiva. Sin bugs conocidos críticos.
Patch / Hotfix	Todos los usuarios	Actualización post-lanzamiento para corregir bugs críticos o vulnerabilidades de seguridad.

BLOQUE 3 · QA · QC · QE

Tres disciplinas distintas que comparten el objetivo de mejorar la calidad del software, pero difieren en su enfoque y momento de actuación.

	QA — Quality Assurance	QC — Quality Control	QE — Quality Engineering
Significado	Aseguramiento de la Calidad	Control de la Calidad	Ingeniería de la Calidad
Enfoque	Preventivo. Garantizar buenos procesos para que los defectos no aparezcan.	Reactivo/detectivo. Identificar defectos en el producto ya construido.	Habilitador. Diseñar la infraestructura y herramientas que hacen posible la calidad continua.
¿Cuándo actúa?	Durante todo el ciclo de vida del desarrollo (SDLC).	Al final de cada fase o sprint, antes de liberar.	Transversal. Integrado desde el diseño hasta CI/CD.
Actividades	Definir estándares, auditar procesos, revisar requisitos, formar al equipo.	Ejecutar pruebas, reportar bugs, verificar correcciones, hacer seguimiento de defectos.	Automatización de pruebas, pipelines CI/CD, análisis de cobertura, observabilidad.
Rol típico	QA Analyst, QA Manager	QC Tester, Manual Tester	SDET, Automation Engineer
Producto final	Planes de calidad, estándares, políticas.	Informes de pruebas, listados de bugs.	Frameworks de automatización, dashboards de calidad.
Analogía	El arquitecto que diseña normas de construcción para evitar problemas.	El inspector que revisa el edificio y encuentra grietas.	El ingeniero que crea las máquinas para construir mejor y más rápido.

En resumen: QA establece el marco y los procesos; QC aplica esos procesos verificando el producto; QE automatiza y escala la capacidad de verificación para entornos de entrega continua. Las tres son complementarias.

BLOQUE 4 · UI / UX

4.1 UI — User Interface (Interfaz de Usuario)

Todo lo que el usuario **ve e interactúa visualmente** en pantalla: colores, tipografía, botones, layouts y animaciones. Su objetivo es que la interfaz sea atractiva, coherente y accesible.

4.2 UX — User Experience (Experiencia de Usuario)

Cómo el usuario **siente e interpreta** su interacción con el producto. Abarca toda la experiencia: desde que el usuario llega hasta que logra su objetivo. Su fin es que el producto sea útil, usable y satisfactorio.

4.3 Diferencias UI vs UX

UI	UX
Enfoque visual y estético.	Enfoque funcional y emocional.
¿Se ve bien? ¿Es coherente?	¿Es fácil de usar? ¿Resuelve el problema?
Actúa tras definir estructura y flujos.	Actúa desde la fase de investigación inicial.
Primera impresión y percepción de calidad.	Retención y satisfacción a largo plazo.
Producto: pantallas y componentes visuales.	Producto: wireframes, prototipos y flujos de usuario.

Relación: UI es la capa visible de lo planificado en UX. Sin UX, la UI es bonita pero frustrante. Sin UI, la UX es funcional pero pobre visualmente.

4.4 VSCode vs VSCodium

Dos editores de código con una base compartida pero filosofías distintas.

VSCode	VSCodium
Desarrollado por Microsoft.	Compilado por la comunidad open-source.
Binario con componentes propietarios de Microsoft.	100% open-source (MIT). Sin componentes propietarios.
Envía telemetría y datos de uso a Microsoft.	Telemetría eliminada por completo.
Marketplace oficial de Microsoft. Máximo ecosistema.	Usa Open VSX Registry. Algunas extensiones exclusivas de Microsoft no están disponibles.
GitHub Copilot, Azure, Live Share y Settings Sync integrados.	Funciones dependientes de servicios Microsoft no disponibles o limitadas.
Más documentación y soporte comunitario.	Algo más ligero al no cargar componentes extra.

Analogía: La relación VSCode / VSCodium es similar a Chrome / Chromium. Comparten casi todo el código pero uno incluye componentes propietarios y el otro no.